

SGA Web con PrimeFaces y Jakarta EE



✦ Guía Paso a Paso: Implementación de PrimeFaces en una Aplicación Java EE 🚀

1 Introducción

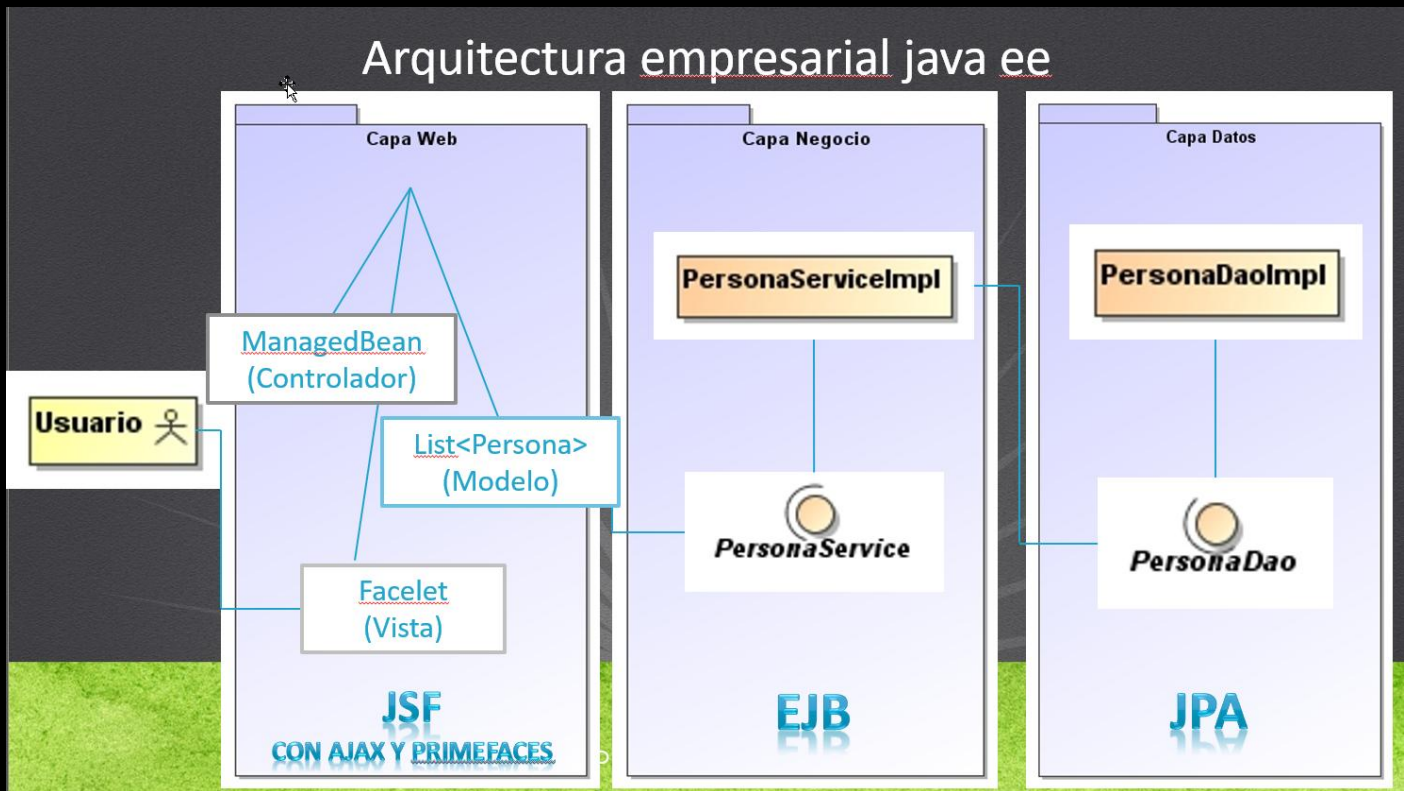
En esta guía, vamos a integrar **PrimeFaces** en un proyecto Java EE basado en **JSF**, **EJB** y **JPA**. La finalidad es mejorar la interfaz de usuario con componentes avanzados y facilitar la interacción con la base de datos mediante AJAX.

✦ Objetivo:

- Agregar la tecnología **JSF con PrimeFaces** a nuestro proyecto SGA.
- Implementar una tabla dinámica para listar y modificar registros de personas.
- Manejar eventos AJAX para una mejor experiencia de usuario.
- Integrar validaciones y operaciones CRUD con EJB.

✦ Arquitectura utilizada:

- **Capa Web:** JSF (PrimeFaces), ManagedBean (Controlador)
- **Capa de Negocio:** EJB (PersonaServiceImpl, PersonaService)
- **Capa de Datos:** JPA (PersonaDaoImpl, PersonaDao)



2 Configuración del Proyecto

Vamos a configurar el proyecto para integrar **PrimeFaces** y preparar la estructura.

◆ Paso 1: Agregar la Dependencia de PrimeFaces

En el archivo `pom.xml`, agregamos la siguiente dependencia:

```

<dependency>
  <groupId>org.primefaces</groupId>
  <artifactId>primefaces</artifactId>
  <version>15.0.0</version>
  <classifier>jakarta</classifier>
</dependency>
  
```

◆ Explicación:

Esto permite utilizar los componentes de **PrimeFaces** en nuestro proyecto.

◆ Código `pom.xml` completo:

```

<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  
```

```
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>sga</groupId>
  <artifactId>sga-jee-web</artifactId>
  <version>1.0</version>
  <packaging>war</packaging>
  <name>sga-jee-web</name>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <jakartaee>11.0.0-M1</jakartaee>
  </properties>

  <dependencies>
    <dependency>
      <groupId>jakarta.platform</groupId>
      <artifactId>jakarta.jakartaee-api</artifactId>
      <version>${jakartaee}</version>
      <scope>provided</scope>
    </dependency>
    <dependency>
      <groupId>jakarta.persistence</groupId>
      <artifactId>jakarta.persistence-api</artifactId>
      <version>3.2.0</version>
    </dependency>
    <dependency>
      <groupId>org.eclipse.persistence</groupId>
      <artifactId>eclipselink</artifactId>
      <version>4.0.5</version>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>9.2.0</version>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>2.0.16</version>
      <type>jar</type>
    </dependency>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
```

```

        <version>2.0.16</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.primefaces</groupId>
        <artifactId>primefaces</artifactId>
        <version>15.0.0</version>
        <classifier>jakarta</classifier>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.12.1</version>
            <configuration>
                <source>21</source>
                <target>21</target>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-war-plugin</artifactId>
            <version>3.4.0</version>
        </plugin>
    </plugins>
</build>
</project>

```

◆ Paso 2: Configurar el archivo `web.xml`

✦ **Modificamos `web.xml` para definir la página de inicio:**

```

<context-param>
    <param-name>jakarta.faces.PROJECT_STAGE</param-name>
    <param-value>Development</param-value>
</context-param>

<welcome-file-list>
    <welcome-file>index.xhtml</welcome-file>
</welcome-file-list>

```

◆ Explicación:

1. El parámetro `PROJECT_STAGE` define el entorno en el que se ejecuta la aplicación JSF y permite que la configuración se adapte según el estado del proyecto.

🔗 ¿Por qué es importante?

1. **Mejor depuración** 🛠️ → En `Development`, se muestran más detalles en logs y errores de JSF.
 2. **Mayor seguridad en producción** 🔒 → En `Production`, los mensajes de error no se muestran al usuario final.
 3. **Optimización automática** 🚀 → JSF aplica **cambios internos** para mejorar el rendimiento en producción.
2. Esto establece que cuando se acceda a la aplicación, se cargará `index.xhtml` como página principal.

◆ Código `web.xml` completo:

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee https://jakarta.ee/xml/ns/jakartaee/web-
app_6_0.xsd"
  version="6.0">

  <context-param>
    <param-name>jakarta.faces.PROJECT_STAGE</param-name>
    <param-value>Development</param-value>
  </context-param>

  <welcome-file-list>
    <welcome-file>index.xhtml</welcome-file>
  </welcome-file-list>
</web-app>
```

3 Creación de la Interfaz de Usuario con JSF y PrimeFaces

- ◆ Eliminamos cualquier archivo `.html` o `.jsp` que ya no vamos a utilizar, como `index.html`, etc.
- ◆ Paso 3: Crear `index.xhtml`

✂️ Este archivo es la pantalla de inicio con un botón para listar personas.

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
  xmlns:h="jakarta.faces.html">
  <h:head>
```

```

        <title>Sistema SGA</title>
    </h:head>
    <h:body>
        <h2>Sistema SGA</h2>
        <h:form>
            <h:commandButton value="Listar Personas" action="listarPersonas" />
        </h:form>
    </h:body>
</html>

```

◆ Explicación:

- Define la estructura básica de la aplicación.
- Incluye un botón que redirige a la página `listarPersonas.xhtml`.

◆ Paso 4: Crear `listarPersonas.xhtml`

✦ Esta página mostrará la lista de personas.

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:p="primefaces">
    <h:head>
        <title>Listado de Personas</title>
    </h:head>
    <h:body>
        <h:form>
            <h2 style="text-align:center; margin-bottom:15px;">Listado de Personas</h2>
            <p:dataTable value="#{personaBean.personas}" var="persona">
                <p:column headerText="Id">#{persona.idPersona}</p:column>
                <p:column headerText="Nombre">#{persona.nombre}</p:column>
                <p:column headerText="Apellido">#{persona.apellido}</p:column>
                <p:column headerText="Email">#{persona.email}</p:column>
                <p:column headerText="Teléfono">#{persona.telefono}</p:column>
            </p:dataTable>
        </h:form>
    </h:body>
</html>

```

◆ Explicación:

- Muestra la lista de personas en una tabla.

Implementación del Managed Bean

- ♦ Eliminamos cualquier archivo .java que ya no vamos a utilizar en el paquete sga.web como los Servlets.
- ♦ Paso 5: Crear `PersonaBean.java`

✖ Este Managed Bean maneja la lógica de presentación en la capa web.

```
package sga.web;

import java.util.List;
import jakarta.annotation.PostConstruct;
import jakarta.enterprise.context.RequestScoped;
import jakarta.inject.Inject;
import jakarta.inject.Named;
import sga.domain.Persona;
import sga.servicio.PersonaService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Named("personaBean")
@RequestScoped
public class PersonaBean {

    private static final Logger log = LoggerFactory.getLogger(PersonaBean.class);

    @Inject
    private PersonaService personaService;

    List<Persona> personas;

    @PostConstruct
    public void inicializar(){
        //Iniciamos las variables
        this.personas = personaService.listarPersonas();
        log.debug("personas recuperadas en ManagedBean:" + this.personas);
    }

    public List<Persona> getPersonas() {
        return personas;
    }
}
```

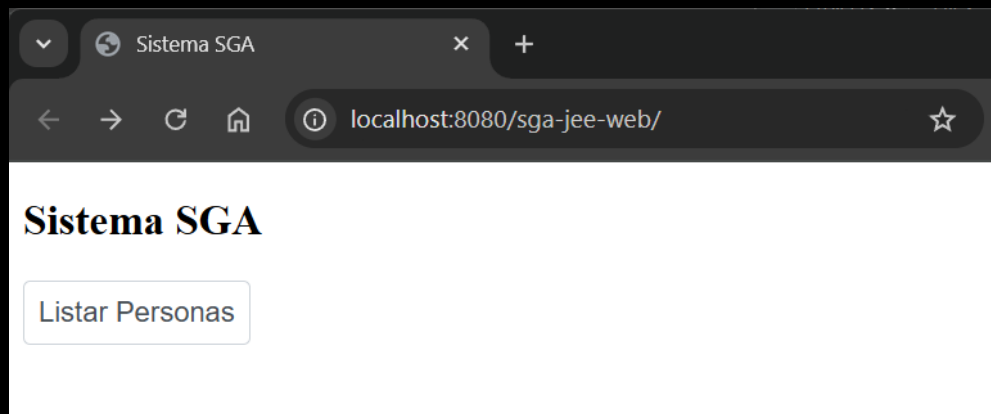
```
public void setPersonas(List<Persona> personas) {  
    this.personas = personas;  
}  
  
}
```

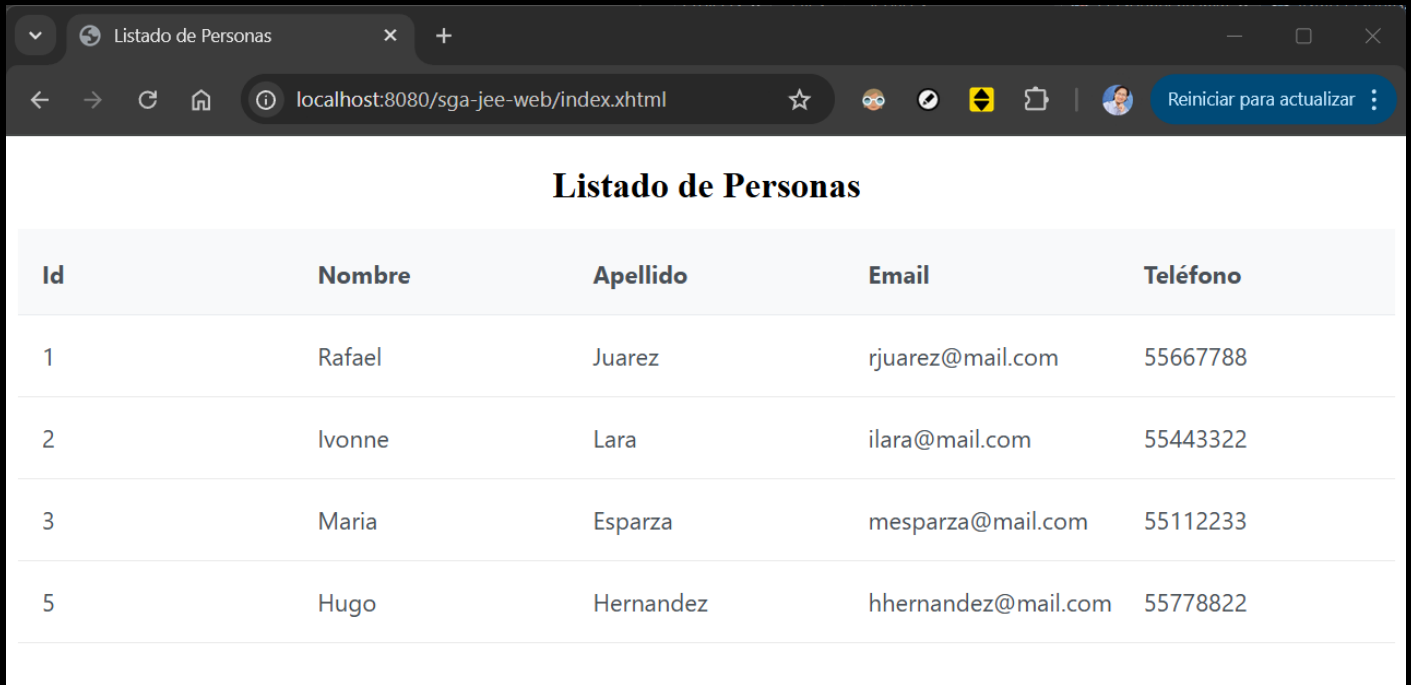
◆ Explicación:

- `@Named("personaBean")`: Define el Managed Bean en JSF.
- `@PostConstruct`: Carga la lista de personas al iniciar.

5 Ejecución

- Detenemos Glassfish, compilamos y volvemos a ejecutar nuestra aplicación.
- Entramos a: <http://localhost:8080/sga-jee-web/index.xhtml>





The screenshot shows a web browser window with the title 'Listado de Personas'. The address bar shows 'localhost:8080/sga-jee-web/index.xhtml'. The page content displays a table with the following data:

Id	Nombre	Apellido	Email	Teléfono
1	Rafael	Juarez	rjuarez@mail.com	55667788
2	Ivonne	Lara	ilara@mail.com	55443322
3	Maria	Esparza	mesparza@mail.com	55112233
5	Hugo	Hernandez	hhernandez@mail.com	55778822

Conclusión

- Hemos integrado **PrimeFaces** en nuestro proyecto Jakarta EE, permitiendo:

 Listar registros desde la base de datos.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)